

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN
THÔNG KHOA ĐÀO TẠO SAU ĐẠI HỌC**



CÁC HỆ THỐNG PHÂN TÁN

**ĐỀ TÀI
ỨNG DỤNG TRÒ CHUYỆN THEO MÔ HÌNH PHÂN TÁN
PEER - TO - PEER (P2P)**

Giảng viên:	TS. Kim Ngọc Bách
Lớp:	M24CQHT02-B
Nhóm:	07
Thành viên nhóm	Nguyễn Đăng Khoa Hoàng Tiến Quyết Lê Đình Hiếu

MỤC LỤC

1. Giới thiệu	3
Mục tiêu chính của đề tài	3
Tính mở rộng	4
Ý nghĩa thực tiễn	4
2. Lý thuyết và Công nghệ sử dụng	
2.1. Mô hình P2P (Peer-to-Peer)	4
2.2. Giao tiếp TCP trong Go	5
2.3. WebSocket và giao diện web	7
2.4. Mô hình Hybrid (TCP + WebSocket)	9
3. Thiết kế hệ thống	9
3.1. Kiến trúc tổng thể hệ thống	9
3.2. Thiết kế chi tiết luồng giao tiếp	11
3.3. Thiết kế WebSocket và giao diện người dùng	13
4. Kết quả thực nghiệm	15
4.1. Trường hợp 2 peer	15
4.2. Trường hợp mở rộng 3 peer	16
5. KẾT LUẬN	19

1. Giới thiệu

Trong thời đại công nghệ hiện nay, các ứng dụng trò chuyện (chat application) đóng vai trò quan trọng trong việc kết nối con người và truyền tải thông tin nhanh chóng. Phần lớn các ứng dụng hiện tại như Zalo, Messenger, Telegram,... đều sử dụng mô hình Client–Server, nơi mọi dữ liệu phải đi qua máy chủ trung tâm để truyền tải giữa các người dùng. Tuy nhiên, mô hình này có một số hạn chế như: chi phí duy trì server, độ trễ khi truy cập qua máy chủ trung gian, và đặc biệt là các vấn đề về quyền riêng tư và bảo mật dữ liệu.

Đề tài này được thực hiện với mục tiêu xây dựng một ứng dụng trò chuyện theo mô hình phân tán peer-to-peer (P2P) – nơi các thiết bị ngang hàng (peer) có thể kết nối trực tiếp với nhau mà không cần máy chủ trung gian. Điều này giúp giảm thiểu chi phí hệ thống, tăng tốc độ truyền tải, và đặc biệt phù hợp với các mô hình mạng cục bộ hoặc ứng dụng phi tập trung (decentralized).

Một đặc điểm nổi bật của đề tài là sự kết hợp giữa giao tiếp TCP truyền thống và giao diện Web hiện đại thông qua WebSocket. Mỗi peer không chỉ đóng vai trò là một thực thể mạng có thể gửi và nhận tin nhắn TCP, mà còn cung cấp giao diện đồ họa trên trình duyệt, giúp người dùng có thể trò chuyện dễ dàng mà không cần thao tác dòng lệnh.

Mục tiêu chính của đề tài bao gồm:

- Xây dựng ứng dụng chat giữa 2 peer sử dụng giao thức TCP.
- Tích hợp giao diện Web bằng WebSocket để người dùng có thể tương tác qua trình duyệt.
- Mỗi peer đóng vai trò vừa là TCP server (lắng nghe), vừa là TCP client (kết nối đến peer khác).
- Tin nhắn được truyền đi trực tiếp giữa các peer, không thông qua máy chủ trung tâm.
- Hiện thị tên người gửi và nội dung tin nhắn trên giao diện.

Tính mở rộng:

- Mặc dù hệ thống ban đầu được thiết kế với 2 peer, nhưng mã nguồn và kiến trúc đã được điều chỉnh để cho phép kết nối đồng thời nhiều peer (ví dụ 3 hoặc hơn). Trong mô hình mở rộng này, khi một peer gửi tin nhắn, nội dung sẽ được chuyển tới toàn bộ các peer còn lại đang kết nối, mô phỏng một hệ thống chat nhóm phân tán đơn giản.

Ý nghĩa thực tiễn: Hệ thống này phù hợp với các tình huống:

- Truyền thông nội bộ trong mạng LAN không có internet.
- Các ứng dụng thử nghiệm mạng phân tán hoặc đào tạo công nghệ P2P.
- Xây dựng ứng dụng trò chuyện offline, bảo mật, không phụ thuộc vào bên thứ ba.

→ Với cấu trúc đơn giản, rõ ràng và mã nguồn ngắn gọn, đề tài là một ví dụ thực hành hữu ích cho sinh viên học về lập trình mạng, hệ thống phân tán và ứng dụng thời gian thực

2. Lý thuyết và Công nghệ sử dụng

2.1. Mô hình P2P (Peer-to-Peer)

Khái niệm

- Mô hình Peer-to-Peer (P2P) là một kiến trúc mạng trong đó mỗi nút (node) trong hệ thống đóng vai trò vừa là máy khách (client) vừa là máy chủ (server). Điều này có nghĩa là các peer có thể vừa gửi, vừa nhận dữ liệu trực tiếp từ nhau mà không cần thông qua một máy chủ trung tâm.
- Khác với mô hình Client–Server truyền thống – nơi một máy chủ tập trung đóng vai trò trung gian xử lý và phân phối thông tin – mô hình P2P cho phép các peer giao tiếp trực tiếp, từ đó giảm thiểu độ trễ, tránh tình trạng quá tải tại máy chủ, và tăng tính phân tán cho hệ thống.

Đặc điểm chính của mô hình P2P:

Đặc điểm	Mô tả
Không có máy chủ trung tâm	Các peer kết nối trực tiếp với nhau mà không qua trung gian.
Phân tán	Tài nguyên và dữ liệu được chia sẻ giữa các peer thay vì tập trung.
Mỗi peer có quyền ngang nhau	Không có sự phân cấp rõ ràng giữa các nút trong mạng.
Tăng khả năng mở rộng	Dễ dàng mở rộng số lượng peer mà không ảnh hưởng đến cấu trúc chung.
Khả năng chống lỗi tốt hơn	Khi một peer gặp sự cố, các peer khác vẫn hoạt động bình thường.

Lợi ích của mô hình P2P trong ứng dụng chat:

- Không cần máy chủ → Giảm chi phí vận hành.
- Truyền tin trực tiếp → Tốc độ phản hồi nhanh, độ trễ thấp.
- Tính riêng tư cao → Dữ liệu không đi qua bên thứ ba.
- Tính mở rộng linh hoạt → Có thể kết nối thêm peer mới bất kỳ lúc nào.

Ứng dụng thực tế của P2P:

- Truyền tải dữ liệu: BitTorrent, IPFS, Resilio Sync.
- Trò chuyện và gọi video: Skype (phiên bản đầu), Tox, Jami.
- Blockchain: Bitcoin, Ethereum – nơi mỗi nút là một peer lưu trữ sổ cái phân tán.

Mô hình sử dụng trong đề tài: Trong đề tài này, mỗi peer hoạt động đồng thời như:

- Một TCP server để lắng nghe tin nhắn đến từ peer khác.
- Một TCP client để chủ động kết nối và gửi tin nhắn đi.

Kết nối giữa các peer được thực hiện qua giao thức TCP. Giao diện người dùng được hiển thị thông qua WebSocket nội bộ – giúp thao tác gửi/nhận tin nhắn trở nên thân thiện và realtime hơn.

2.2. Giao tiếp TCP trong Go

Khái quát

- Golang (Go) là một ngôn ngữ lập trình mạnh mẽ được thiết kế bởi Google, nổi bật với khả năng xử lý đồng thời (concurrency) và lập trình mạng hiệu quả. Trong ứng dụng này, Go được sử dụng để thiết lập kết nối TCP (Transmission Control Protocol) giữa các peer – cho phép truyền tin nhắn theo thời gian thực, đảm bảo độ tin cậy, thứ tự và toàn vẹn dữ liệu.
- Giao thức TCP được lựa chọn thay vì UDP bởi vì:
 - + Đảm bảo tin nhắn được nhận đúng thứ tự
 - + Phát hiện và khôi phục gói tin bị mất
 - + Phù hợp với ứng dụng nhắn tin cần độ tin cậy cao

Các thư viện và thành phần chính trong Go

- Go cung cấp sẵn thư viện `net` để làm việc với các giao thức mạng như TCP và UDP. Trong chương trình, ta sử dụng một số hàm và cấu trúc dữ liệu quan trọng như sau:

Hàm / Gói	Mục đích
<code>net.Listen("tcp", addr)</code>	Tạo TCP server, lắng nghe kết nối đến tại địa chỉ <code>addr</code>
<code>net.Dial("tcp", addr)</code>	Tạo TCP client, chủ động kết nối đến peer tại địa chỉ <code>addr</code>
<code>conn.Read()</code>	Đọc dữ liệu từ kết nối TCP
<code>conn.Write()</code> hoặc <code>fmt.Fprintf(conn, ...)</code>	Gửi dữ liệu qua kết nối TCP
<code>conn.Close()</code>	Đóng kết nối TCP
<code>bufio.NewScanner(conn)</code>	Đọc dữ liệu từng dòng từ kết nối TCP

Cách triển khai trong chương trình

Trong hệ thống chat P2P, mỗi peer vừa là server vừa là client, cụ thể:

a. Peer là TCP Server – dùng để lắng nghe tin nhắn

```
ln, err := net.Listen("tcp", ":"+port)
conn, err := ln.Accept()
go readFromPeer(conn) // nhận và xử lý tin nhắn đến
```

b. Peer là TCP Client – dùng để kết nối đến peer khác

```
conn, err := net.Dial("tcp", "127.0.0.1:8001")
peerConns = append(peerConns, conn)
```

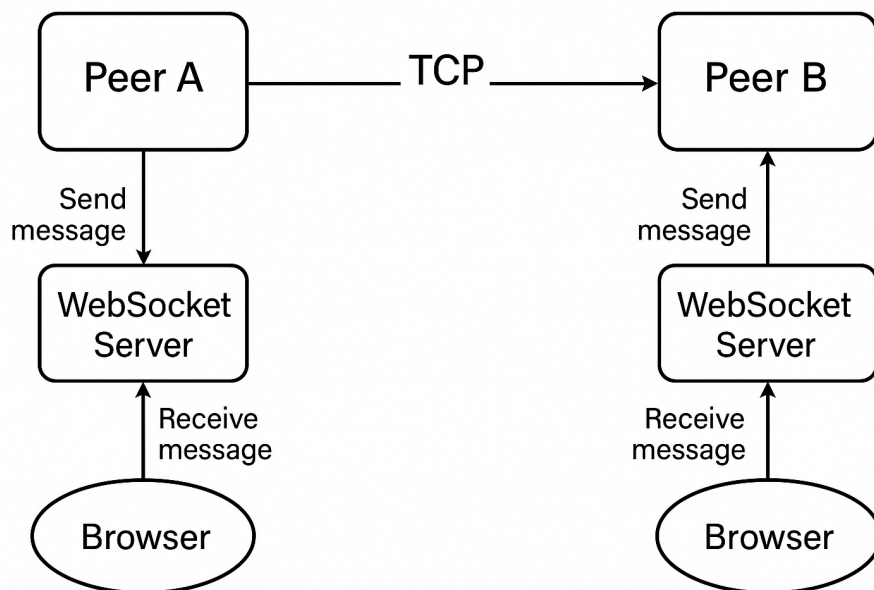
c. Gửi tin nhắn đến peer

```
for _, conn := range peerConns {
    fmt.Fprintf(conn, "%s: %s\n", peerName, msg.Message)
}
```

d. Nhận và xử lý tin nhắn đến

```
scanner := bufio.NewScanner(conn)
for scanner.Scan() {
    line := scanner.Text()
    var msg Message
    json.Unmarshal([]byte(line), &msg)
    broadcast <- msg
}
```

Mô hình kết nối TCP



Cả Peer A và Peer B đều:

- Lắng nghe TCP để nhận tin nhắn
- Chủ động kết nối TCP đến peer kia để gửi tin nhắn

Tính năng nổi bật khi dùng TCP trong Go

- Dễ dàng xây dựng kiến trúc song song (concurrent) nhờ goroutine
- Mỗi peer có thể mở nhiều kết nối TCP cùng lúc
- Tích hợp tốt với WebSocket để đồng bộ giao diện

2.3. WebSocket và giao diện web

Giới thiệu về WebSocket

- WebSocket là một giao thức truyền thông hai chiều (full-duplex) chạy trên một kết nối TCP duy nhất, cho phép client và server duy trì kết nối liên tục để trao đổi dữ liệu theo thời gian thực (real-time). Không giống như HTTP – vốn là giao thức một chiều, nơi client phải liên tục gửi yêu cầu để nhận dữ liệu – WebSocket cho phép server đẩy dữ liệu đến client ngay khi có sự kiện xảy ra.
- WebSocket được dùng để tạo một giao diện người dùng (UI) cho phép người dùng nhập và xem tin nhắn trò chuyện trong môi trường trình duyệt, đồng thời kết nối trực tiếp với phần lõi của ứng dụng thông qua WebSocket.

Cơ chế hoạt động WebSocket trong hệ thống

- Khi trình duyệt mở:
 - + Gửi yêu cầu WebSocket đến địa chỉ `ws://localhost:<cổng>/ws`.
 - + Kết nối này được nâng cấp từ HTTP lên WebSocket.
- Người dùng gửi tin nhắn:
 - + Giao diện Web thu thập nội dung tin nhắn từ ô nhập liệu.
 - + Tin nhắn được đóng gói dưới dạng JSON và gửi đến server WebSocket.
- Server WebSocket xử lý:
 - + Nhận JSON từ client → phân tích nội dung.
 - + Gửi tin nhắn đến các peer khác qua TCP.
 - + Đồng thời, phát tán lại (broadcast) cho các client WebSocket đang kết nối.
- Giao diện người dùng cập nhật:
 - + Tin nhắn được hiển thị lên vùng chat.
 - + Hiển thị rõ ràng tên người gửi và nội dung.

Ưu điểm khi dùng WebSocket

Tính năng	Lợi ích cụ thể trong ứng dụng
Real-time	Tin nhắn xuất hiện tức thì không cần tải lại.
Hai chiều (full-duplex)	Cho phép gửi và nhận tin nhắn đồng thời.
Tiết kiệm băng thông	Không cần HTTP polling liên tục.

Dễ tích hợp giao diện web	Tương thích với JavaScript, HTML dễ triển khai.
---------------------------	---

HTML giao diện chat: Phần giao diện web được viết trực tiếp trong mã nguồn Go bằng `http.HandleFunc("/")`, bao gồm các thành phần chính:

```
<div id="chat"></div>

<input id="message" placeholder="Nội dung tin nhắn">

<button onclick="sendMsg()">Gửi</button>
```

Và phần JavaScript sử dụng WebSocket như sau:

```
let ws = new WebSocket("ws://" + location.host + "/ws");

ws.onmessage = (event) => {

    let msg = JSON.parse(event.data);

    let div = document.createElement("div");

    div.innerText = msg.name + ": " + msg.message;

    document.getElementById("chat").appendChild(div);

};
```

2.4. Mô hình Hybrid (TCP + WebSocket)

Trong ứng dụng này, mô hình Hybrid được sử dụng nhằm kết hợp ưu điểm của giao tiếp TCP thuần (trong mô hình P2P truyền thống) và giao tiếp WebSocket qua trình duyệt để tạo ra một hệ thống trò chuyện vừa tương tác thời gian thực, vừa có thể tích hợp giao diện người dùng.

Mục tiêu của mô hình hybrid

- Tối ưu hóa giao tiếp giữa các peer: TCP đảm bảo truyền tin ổn định giữa các máy (peers), kể cả khi không có trình duyệt.
- Hỗ trợ giao diện web: WebSocket cho phép người dùng tương tác qua trình duyệt một cách mượt mà, phản hồi thời gian thực mà không cần tải lại trang.
- Tạo mô hình dễ kiểm thử và triển khai: Kết hợp này giúp dễ dàng mô phỏng và vận hành mạng P2P đơn giản trên một máy hoặc nhiều máy nội bộ.

Cấu trúc mô hình

- TCP Layer: Mỗi peer là một TCP server và đồng thời có thể kết nối đến nhiều peer khác (client), tạo thành mạng lưới P2P thực thụ.

- WebSocket Layer: Một server WebSocket chạy nội bộ tại mỗi peer, tiếp nhận và gửi tin nhắn đến trình duyệt đang mở.
- Bộ xử lý trung gian: Các tin nhắn từ giao diện WebSocket được chuyển tới các peer khác qua TCP, và ngược lại, các tin nhận từ peer được hiển thị trên giao diện thông qua broadcast nội bộ.

Ưu điểm của mô hình hybrid

Ưu điểm	Mô tả
Tính linh hoạt	Có thể mở rộng với nhiều giao diện khác nhau (CLI, GUI, Web...)
Truyền thông ổn định	Sử dụng TCP cho phép kết nối ổn định và tin cậy
Hỗ trợ tương tác người dùng	WebSocket giúp xây dựng giao diện web realtime không cần reload
Giao tiếp hai chiều hoàn chỉnh	Trình duyệt ↔ WebSocket ↔ Peer ↔ Peer khác

Ví dụ hoạt động

Giả sử có ba peer: Alice, Bob, và Carol. Mỗi người khởi chạy một peer với địa chỉ và cổng riêng. Giao diện web của mỗi người hiển thị đúng tên peer tương ứng. Khi Alice gửi tin nhắn từ trình duyệt:

- Tin được gửi qua WebSocket đến peer Alice.
- Peer Alice gửi tin qua TCP đến Bob và Carol.

3. Thiết kế hệ thống

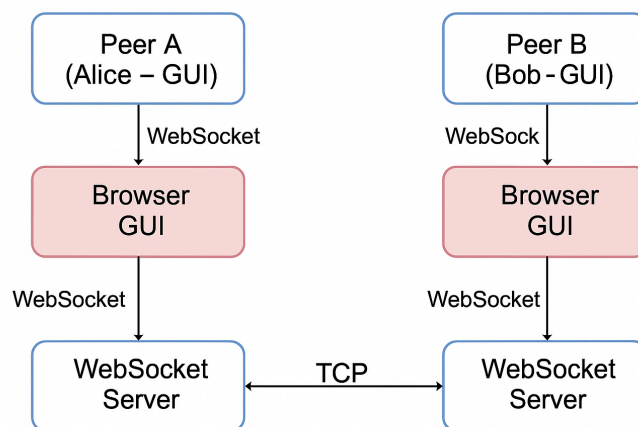
3.1. Kiến trúc tổng thể hệ thống

Mục tiêu thiết kế

- Ứng dụng được xây dựng theo mô hình hybrid peer-to-peer (P2P) với mục tiêu kết hợp:

- + Giao tiếp trực tiếp giữa các peer thông qua kết nối TCP (để duy trì tính chất P2P thật sự).
 - + Giao diện người dùng web-based sử dụng WebSocket để hỗ trợ hiển thị và tương tác dễ dàng.
- Mỗi peer trong hệ thống vừa là một TCP server (để lắng nghe kết nối từ các peer khác), vừa là một TCP client (để chủ động kết nối tới các peer khác). Ngoài ra, mỗi peer cũng chạy một WebSocket server nội bộ để phục vụ giao diện người dùng hiển thị tin nhắn theo thời gian thực.

Mô hình hoạt động tổng quan



- Mỗi peer có thể vừa gửi vừa nhận dữ liệu từ nhiều peer khác.
- Mỗi giao diện người dùng (trình duyệt) kết nối qua WebSocket đến chính peer local tương ứng.
- Tất cả tin nhắn được chuyển tiếp qua các peer TCP, sau đó hiển thị trong giao diện web bằng WebSocket.

Các thành phần chính trong mỗi peer

Thành phần	Vai trò
TCP Server	Lắng nghe kết nối đến từ các peer khác
TCP Client	Kết nối đến địa chỉ IP các peer khác
WebSocket Server	Kết nối tới giao diện frontend

Thành phần	Vai trò
TCP Server	Lắng nghe kết nối đến từ các peer khác
Frontend HTML	Cho phép nhập/xem tin nhắn thời gian thực
Broadcast Handler	Gửi tin nhắn đến tất cả peer + frontend
Peer Connection Handler	Đọc tin nhắn từ peer, đưa vào broadcast

Luồng xử lý dữ liệu

1. Người dùng nhập tin nhắn trong giao diện web.
2. Tin nhắn gửi qua WebSocket đến WebSocket server nội bộ.
3. Server:
 - Phát tin nhắn sang tất cả các peer qua TCP.
 - Đồng thời phát lại tin nhắn tới tất cả frontend WebSocket đang kết nối.
4. Các peer nhận được tin nhắn qua TCP:
 - Đọc nội dung.
 - Phát lại lên giao diện frontend WebSocket tương ứng của mình.

3.2. Thiết kế chi tiết luồng giao tiếp

Để đảm bảo giao tiếp hai chiều đồng thời giữa các peer và giao diện người dùng, ứng dụng sử dụng luồng giao tiếp kết hợp giữa TCP và WebSocket như sau:

(1) Giao tiếp giữa các peer (TCP layer)

- Mỗi peer thực hiện đồng thời 2 chức năng:
 - TCP server: Lắng nghe kết nối đến từ peer khác (qua cổng TCP riêng).
 - TCP client: Tự động kết nối tới các peer khác được chỉ định khi khởi động.
- Sau khi thiết lập kết nối, các peer có thể:
 - Gửi tin nhắn đến tất cả các peer mà nó đang kết nối (qua `[]net.Conn`).
 - Nhận tin nhắn từ các peer khác qua `conn.Read()` và truyền về WebSocket frontend.

Các thông điệp được trao đổi ở định dạng JSON và tuân theo cấu trúc:

```

{
  "name": "Tên peer gửi",
  "message": "Nội dung tin nhắn"
}

```

(2) Giao tiếp giữa peer và người dùng giao diện web (WebSocket layer)

- Mỗi peer khởi động 1 HTTP server (port riêng, ví dụ :8080) để:
 - Phục vụ frontend HTML
 - Thiết lập WebSocket server tại endpoint /ws
- Khi người dùng trên giao diện web gửi tin nhắn:
 - Tin nhắn được gửi từ trình duyệt thông qua WebSocket (JavaScript ws.send()).
 - Server Go nhận được msg, gửi broadcast tới tất cả các peer đang kết nối TCP.
 - Đồng thời, server cũng hiển thị lại tin nhắn tại giao diện hiện tại (qua channel broadcast).
- Khi peer nhận tin nhắn từ peer khác (TCP layer):
 - Tin nhắn được giải mã, gửi vào channel broadcast, và gửi đến các client WebSocket.

(3) Tóm tắt mô hình luồng giao tiếp

Thành phần	Vai trò	Giao tiếp
Peer A	Gửi & nhận TCP + WebSocket	Kết nối đến Peer B, giao diện A
Peer B	Gửi & nhận TCP + WebSocket	Lắng nghe từ A, kết nối đến C, giao diện B
Peer C	Gửi & nhận TCP + WebSocket	Lắng nghe từ B, giao diện C
Frontend Web (HTML)	Tương tác người dùng	Giao tiếp với peer qua WebSocket

Ví dụ chuỗi luồng tin nhắn

- + Người dùng A (trên localhost:8080) gửi tin nhắn "Hi" → WebSocket → Peer A

- + Peer A chuyển tin nhắn tới B và C qua TCP
- + Peer B/C nhận tin → Gửi tới giao diện web của mình → hiển thị

3.3. Thiết kế WebSocket và giao diện người dùng

Phần giao diện web của ứng dụng được xây dựng đơn giản nhưng trực quan, sử dụng HTML + JavaScript thuần để tương tác với WebSocket server do mỗi peer khởi tạo. Dưới đây là phân tích chi tiết:

- Cấu trúc giao diện HTML

Giao diện được trả về trực tiếp từ hàm `serveFrontend(w http.ResponseWriter, r *http.Request)` trong file `hybrid_p2p_gui_chat.go`, với nội dung HTML như sau:

```
<!DOCTYPE html>

<html>

<head><title>Hybrid P2P Chat</title></head>

<body>

<h2>Hybrid P2P Chat</h2>

<div id="chat" style="height:300px;overflow:auto;border:1px solid
#ccc;padding:10px;"></div>

<input id="message" placeholder="Nội dung tin nhắn" style="width:80%">

<button onclick="sendMsg()">Gửi</button>

</body>

</html>
```

- `<div id="chat">`: Nơi hiển thị luồng tin nhắn giữa các peer.
- `<input id="message">`: Ô nhập nội dung tin nhắn.
- `<button>`: Gửi tin nhắn.

Luồng xử lý JavaScript và WebSocket

JavaScript tích hợp sẵn trong HTML để:

- Kết nối đến endpoint `/ws` (WebSocket server).
- Nhận và hiển thị tin nhắn từ các peer.
- Gửi tin nhắn từ giao diện người dùng đến WebSocket server nội bộ.

```
let ws = new WebSocket("ws://" + location.host + "/ws");
```

```

ws.onmessage = (event) => {

  let msg = JSON.parse(event.data);

  let div = document.createElement("div");

  div.innerText = '💬' + msg.name + ': ' + msg.message;

  document.getElementById("chat").appendChild(div);

};

function sendMsg() {

  let content = document.getElementById("message").value;

  fetch('/name').then(r => r.text()).then(name => {

    ws.send(JSON.stringify({name: name, message: content}));

    document.getElementById("message").value = "";

  });

}

```

Giải thích:

- new WebSocket(...): kết nối đến WebSocket endpoint /ws đang chạy trên port hiện tại (do peer tạo).
- Khi nhận tin nhắn (onmessage), nội dung được phân tích cú pháp (JSON.parse) và thêm vào phần tử #chat.
- Khi người dùng gửi tin (sendMsg()), đoạn script:
 - Lấy tên peer từ endpoint /name.
 - Tạo JSON gồm name và message.
 - Gửi qua WebSocket tới peer.

- Ưu điểm của thiết kế WebSocket này

- + Realtime: Tin nhắn được truyền và nhận ngay lập tức.
- + Đa người dùng: Mỗi peer có thể phục vụ nhiều client frontend cùng lúc (qua WebSocket).
- + Không cần reload: Giao tiếp xảy ra trong nền, không cần refresh trang.
- + Tích hợp chặt với TCP: WebSocket gửi và nhận tin từ giao thức TCP giữa các peer, đảm bảo mô hình giao tiếp đồng bộ.

- Ví dụ hoạt động

Giả sử Peer A chạy tại localhost:8080, người dùng truy cập trình duyệt và gửi "Hello".

- JavaScript gửi message {name: "Alice", message: "Hello"} qua WebSocket.
- Peer A nhận và:
 - Gửi nội dung đó tới các peer đang kết nối TCP.
 - Gửi lại tin nhắn đó đến chính trình duyệt để hiển thị.

4. Kết quả thực nghiệm

4.1. Trường hợp 2 peer

Để kiểm tra hoạt động của hệ thống trong môi trường thực tế, nhóm tiến hành thực nghiệm với **hai peer** độc lập chạy trên cùng một máy tính (localhost), mỗi peer mở một giao diện trình duyệt riêng để tương tác. Mỗi peer đảm nhận đồng thời 3 vai trò:

- **TCP server**: lắng nghe kết nối từ peer khác.
- **TCP client**: kết nối đến peer đã biết.
- **WebSocket server**: giao tiếp với trình duyệt trên cổng khác.

a) Cách khởi chạy

Terminal 1 (Peer 1 - Alice): `go run hybrid_p2p_gui_chat.go Alice 8000 none 8080`

- Peer này lắng nghe TCP tại cổng 8000, không kết nối tới peer nào khác.
- Giao diện chat hiển thị tại <http://localhost:8080>.

```
PS C:\Users\Asus\go-workspace\p2p-chat\p2p-chat> go run hybrid_p2p_gui_chat.go Alice 8000 none 8080
Peer đang chạy tại: http://localhost:8080
Đang lắng nghe TCP tại cổng: 8000
█
```

Terminal 2 (Peer 2 - Bob): `go run hybrid_p2p_gui_chat.go Bob 8001 127.0.0.1:8000 8081`

- Peer này lắng nghe TCP tại 8001, kết nối tới peer Alice tại 127.0.0.1:8000.
- Giao diện chat hiển thị tại <http://localhost:8081>.

```
PS C:\Users\Asus\go-workspace\p2p-chat\p2p-chat> go run hybrid_p2p_gui_chat.go Bob 8001 127.0.0.1:8000 8081
2025/08/07 11:10:51 Đã kết nối tới peer: 127.0.0.1:8000
Peer đang chạy tại: http://localhost:8081
Đang lắng nghe TCP tại cổng: 8001
█
```

b) Tương tác thử nghiệm

- Người dùng Alice gửi tin nhắn từ giao diện web (8080) → tin nhắn lập tức hiển thị tại cả hai giao diện Alice và Bob.
- Người dùng Bob gửi tin nhắn từ giao diện web (8081) → tin nhắn cũng xuất hiện tức thời tại cả hai peer.
- Mỗi tin nhắn đều hiển thị đúng **tên người gửi** nhờ dữ liệu được đồng bộ giữa WebSocket và TCP.

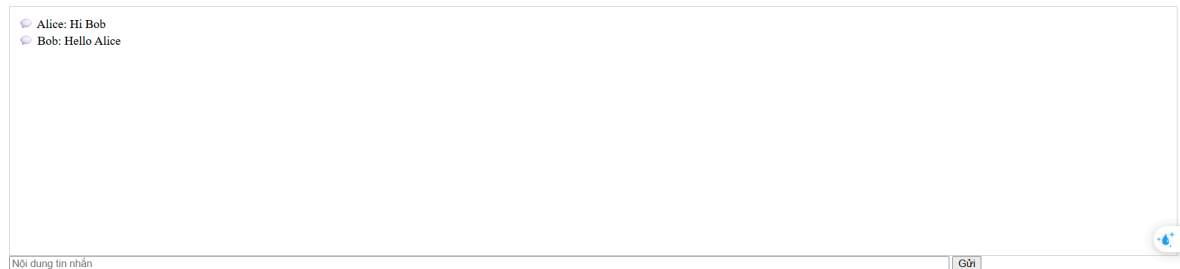
c) Kết quả quan sát

Người gửi	Gửi từ trình duyệt	Cổng giao diện	Peer nhận	Kết quả
Alice	http://localhost:8080	8080	Bob	Tin nhắn hiển thị ngay
Bob	http://localhost:8081	8081	Alice	Tin nhắn hiển thị ngay

d) Giao diện mẫu

Mỗi peer đều có giao diện như sau:

Hybrid P2P Chat (Group 7)



e) Đánh giá

- **Thành công:** Hệ thống hoạt động đúng kỳ vọng trong mô hình 2 peer.
- **Thời gian phản hồi:** Gần như tức thì nhờ sử dụng kết nối liên tục (WebSocket + TCP).
- **Lưu ý:** Địa chỉ và cổng cần được thiết lập chính xác khi chạy chương trình.

4.2. Trường hợp mở rộng 3 peer

Sau khi kiểm thử thành công mô hình 2 peer, nhóm tiến hành **mở rộng hệ thống lên 3 peer**, trong đó mỗi peer vừa lắng nghe kết nối, vừa chủ động kết nối đến các peer còn lại.

a) Cách khởi chạy

Terminal 1 (Peer 1 - Alice): *go run hybrid_p2p_gui_chat.go Alice 8000 none 8080*

- Lắng nghe TCP tại 8000, không kết nối tới peer nào.
- Giao diện Web tại: <http://localhost:8080>

```
PS C:\Users\Asus\go-workspace\p2p-chat\p2p-chat> go run hybrid_p2p_gui_chat.go Alice 8000 none 8080
Peer đang chạy tại: http://localhost:8080
Đang lắng nghe TCP tại cổng: 8000
█
```

Terminal 2 (Peer 2 - Bob): *go run hybrid_p2p_gui_chat.go Bob 8001 127.0.0.1:8000 8081*

- Lắng nghe TCP tại 8001, kết nối tới Alice (127.0.0.1:8000).
- Giao diện Web tại: <http://localhost:8081>

```
PS C:\Users\Asus\go-workspace\p2p-chat\p2p-chat> go run hybrid_p2p_gui_chat.go Bob 8001 127.0.0.1:8000 8081
2025/08/07 11:10:51 Đã kết nối tới peer: 127.0.0.1:8000
Peer đang chạy tại: http://localhost:8081
Đang lắng nghe TCP tại cổng: 8001
█
```

Terminal 3 (Peer 3 - Carol): *go run hybrid_p2p_gui_chat.go Carol 8002 127.0.0.1:8000,127.0.0.1:8001 8082*

- Lắng nghe TCP tại 8002, kết nối tới cả Alice và Bob.
- Giao diện Web tại: <http://localhost:8082>

```
PS C:\Users\Asus\go-workspace\p2p-chat\p2p-chat> go run hybrid_p2p_gui_chat.go Carol 8002 127.0.0.1:8000,127.0.0.1:8001 8082
2025/08/07 11:14:25 Đã kết nối tới peer: 127.0.0.1:8000
2025/08/07 11:14:25 Đã kết nối tới peer: 127.0.0.1:8001
Peer đang chạy tại: http://localhost:8082
Đang lắng nghe TCP tại cổng: 8002
█
```

Lưu ý: Tham số thứ 3 khi chạy Carol là danh sách các địa chỉ TCP peer, cách nhau bởi dấu

b) Tương tác thử nghiệm

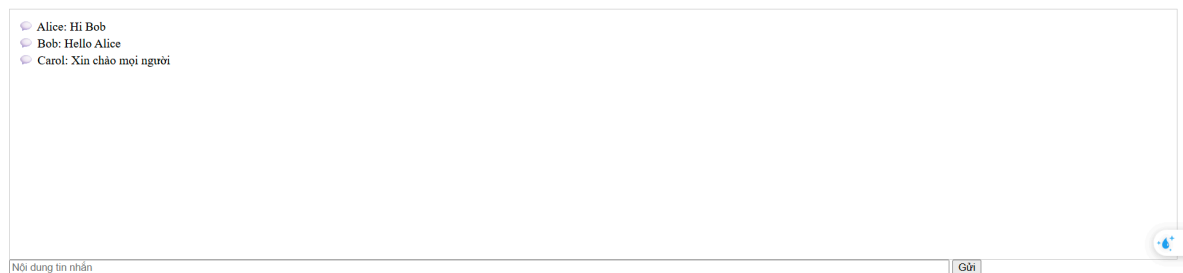
Người gửi	Gửi từ cổng	Peer nhận	Tin nhắn hiển thị
Alice	8080	Bob, Carol	Có

Bob	8081	Alice, Carol	Có
Carol	8082	Alice, Bob	Có

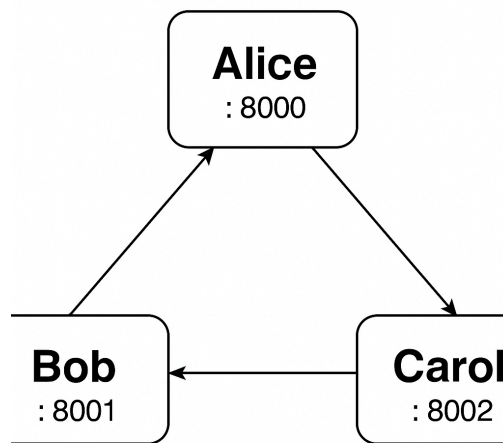
Ví dụ:

- Carol gửi: **“Xin chào mọi người!”** từ `http://localhost:8082`
→ Tin nhắn hiển thị lập tức trên cả Bob (8081) và Alice (8080).

Hybrid P2P Chat (Group 7)



c) Minh họa mô hình kết nối



Carol kết nối đến cả Bob và Alice, Bob kết nối đến Alice. Nhờ đó, tin nhắn có thể được truyền tới mọi peer.

d) Đánh giá

- Hệ thống mở rộng tốt lên nhiều peer mà không thay đổi kiến trúc tổng thể.
- Tin nhắn được đồng bộ giữa tất cả các peer trong mạng.
- Số lượng kết nối TCP tăng tuyến tính theo số peer → cần tối ưu nếu mở rộng lớn hơn (dùng định tuyến hoặc broadcast).

5. KẾT LUẬN

Trong khuôn khổ bài báo cáo này, chúng tôi đã xây dựng và thử nghiệm thành công một ứng dụng trò chuyện ngang hàng (Peer-to-Peer) tích hợp giao diện đồ họa bằng WebSocket. Hệ thống hỗ trợ mô hình hybrid, kết hợp giữa giao tiếp TCP truyền thống giữa các peer và giao diện Web sử dụng WebSocket nhằm tăng tính trực quan, tiện dụng.

Các kết quả thực nghiệm cho thấy:

- Mô hình 2 peer hoạt động ổn định, cho phép gửi và nhận tin nhắn 2 chiều theo thời gian thực.
- Mở rộng lên 3 peer trở lên vẫn đảm bảo truyền thông giữa các node, dù cần thêm các cải tiến về phát hiện peer động, xử lý lỗi, đồng bộ kết nối...
- Giao diện WebSocket giúp tăng trải nghiệm người dùng, dễ thao tác so với chỉ sử dụng CLI.
- Cấu trúc mã nguồn rõ ràng, dễ mở rộng cho các mục tiêu như mã hóa dữ liệu, quản lý danh tính, tích hợp giao thức NAT traversal...

-> **Tóm lại**, hệ thống là một minh chứng thực tế cho khả năng xây dựng các ứng dụng mạng phân tán với khả năng tích hợp giao diện người dùng thân thiện. Trong tương lai, việc tích hợp thêm các cơ chế đồng bộ peer động, bảo mật dữ liệu và chuyển sang môi trường mạng Internet sẽ là những bước đi tiếp theo nhằm hoàn thiện và nâng cao hệ thống.